Lab 00 — Commented answers

Each answer follows the same scheme: the answer, the mechanism, the nuance or the pitfall, an example you can verify at the terminal.

Question 1 — A Linux binary runs everywhere… except on Windows

Answer. A binary asks nothing of "Ubuntu" or "Alpine": it asks everything of the **kernel**, through system calls (`open`, `read`, `fork`…). Those calls are identical on any Linux machine: the distribution only supplies the userland around them. The Windows kernel exposes different, incompatible calls: the binary has no interlocutor there.

Why. The kernel ↔ programs interface is stable and standardized (Linux forbids itself to break it). Everything that distinguishes distributions — package manager, library versions, configuration — lives *above* that interface.

Nuance. "Identical" assumes the required dynamic libraries are present (the `libc` for instance, different between Debian and Alpine — you will face this in lab 05). And WSL does not "translate": WSL 2 runs a **real** Linux kernel inside a VM.

Example.

```
uname -r          # 6.6.87.2-microsoft-standard-WSL2: a real Linux kernel, signed Microsoft
uname -m          # x86_64: the architecture, the other compatibility condition
```

Question 2 — The orphan adopted by PID 1

Answer. The shell (parent of the `sleep`) died with the terminal. The kernel never leaves a process without a parent: the orphan is **adopted** by PID 1 (`systemd`), hence PPID = 1. `sleep` keeps running normally.

Why. The parent has a precise role: when a child dies, the parent reads its exit code (otherwise the child lingers in "zombie" state). So there must always be a guardian of last resort — one of the responsibilities of PID 1.

Nuance. This is exactly why the PID 1 *of a container* is a serious topic (lab 03): there, your application inherits that guardian role without knowing it, and a PID 1 that doesn't bury its zombies, or has no fallback parent, changes the container's behavior.

Example.

```
sleep 300 &
ps -o pid,ppid,cmd | grep [s]leep    #  2419  2363  sleep 300  (PPID = your bash)
# close the terminal, open a new one:
ps -ef | grep [s]leep                #  ubuntu  2419  1  ...  sleep 300  (PPID = 1)
```

Question 3 — `Permission denied`, code 126

Answer. The file lacks the execute bit `x`. Confirmation: `ls -l deploy.sh` (you read `-rw-r--r--`, no `x`). Fix: `chmod +x deploy.sh`. Workaround without fixing anything: `bash deploy.sh` — then it is `bash` (executable, itself) that gets launched, the script being just an argument that is read.

Why. Running `./deploy.sh` asks the kernel to **execute this file**; the kernel checks the `x` bit and refuses. Code 126 is the shell convention: "found, but not executable" — to distinguish from 127, "not found".

Nuance. A file created by an editor or downloaded is born `rw-`: execution is a right you add deliberately. In a Dockerfile (lab 04), the `COPY` of a script followed by `RUN ./script.sh` will fail the same way if the `x` bit was missing in your Git repository.

Example.

```
printf '#!/bin/bash\necho hello\n' > deploy.sh
./deploy.sh ; echo $?      # bash: ./deploy.sh: Permission denied ; 126
chmod +x deploy.sh
./deploy.sh                # hello
```

Question 4 — Shell variable vs environment variable

Answer. Line 1: `1:` (empty). Line 2: `2: hello`. Before `export`, `MSG` is only a variable **of the current shell**; the `bash -c` child is born with the inherited environment, where `MSG` does not exist. After `export`, `MSG` enters the environment, and every child receives a copy of it.

Why. At its creation, a process receives a **copy** of its parent's environment, never a reference: it is a one-way inheritance, frozen at launch time.

Nuance. The **single** quotes of `'echo 1: $MSG'` are essential: they prevent your own shell from substituting `$MSG` before launching the child. With double quotes, both lines would print `hello`... but the parent would have done the substitution, not the child. Important corollary: modifying the environment of an **already running** process is impossible — hence the `podman run -e` fixed at startup (lab 08).

Example.

```
MSG=hello
env | grep MSG          # nothing
export MSG
env | grep MSG          # MSG=hello
```

Question 5 — Code 137

Answer. 137 = **128 + 9**: the process died receiving signal number 9, `SIGKILL`. Nobody gave it the slightest chance to stop cleanly. This code is famous because it is the code of a container shot down: by `docker kill`, by a `docker stop` left unanswered after its grace period, or by the kernel's **OOM killer** when memory runs out.

Why. The shell encodes death-by-signal as `128 + number`, to distinguish it from a voluntary `exit n`. `SIGKILL` is never delivered to the process: the kernel removes it directly, executing no cleanup code.

Nuance. Diagnosing a 137 therefore means finding **who** sent the 9: a human, an orchestrator... or the kernel. `dmesg | grep -i "out of memory"` settles the OOM case. You will make this diagnosis on real containers in lab 03.

Example.

```
bash -c 'kill -9 $$' ; echo $?    # 137  ($$ = the PID of the child bash itself)
sleep 300 & kill -9 $! ; wait $! ; echo $?   # 137 as well
```

Question 6 — Polite `kill`, brutal `kill -9`

Answer. `kill` (thus `SIGTERM`) is a **request**: the process receives it, may run its shutdown code — flush its buffers, close its transactions, close its connections — then exit. `kill -9` (`SIGKILL`) is not delivered to the process: the kernel erases it immediately. A database then loses everything not yet written to disk and will have to replay its journal at restart — or even repair corrupted files.

Why. This is precisely the protocol of `docker stop`: send `SIGTERM` to the container's PID 1, grace period (10 s by default), then `SIGKILL` if the process has not complied. An application that ignores `SIGTERM` is therefore **always** killed brutally when the delay expires.

Nuance. `kill -9` has its place: a stuck process that genuinely ignores `SIGTERM`. The correct reflex is the escalation — `kill`, wait, then `kill -9` — never the reverse. Note also that `SIGKILL` can be neither caught nor ignored: it is the only guaranteed recourse.

Example.

```
sleep 300 &
kill %1        # SIGTERM: the job prints "Terminated"
sleep 300 &
kill -9 %1     # SIGKILL: the job prints "Killed"
```

Question 7 — Reading `-rw-r----- root shadow`

Answer. The nine bits split into three triplets: owner `rw-`, group `r--`, others `---`. Your user is neither `root` (the owner) nor a member of the group `shadow`: it falls into "others", who have **no** right at all — hence the refusal. `sudo cat` runs `cat` with UID 0, and the kernel does not apply permission checks to root. Without `sudo`, only root and the members of the group `shadow` (read-only) can read the file.

Why. At each `open`, the kernel compares the UID/GID of the calling **process** with the file's bits: owner first, else group, else "others". The first applicable triplet is the only one applied.

Nuance. `/etc/shadow` contains the password hashes — it is the canonical example file. Note that the "first applicable triplet" rule can surprise: a file `----rw-rw-` would be unreadable… by its own owner.

Example.

```
id                      # uid=1000(ubuntu) ...: not root, not group shadow
cat /etc/shadow         # Permission denied, code 1
sudo head -n 1 /etc/shadow   # root:*:20501:0:99999:7:::
```

Question 8 — Two streams, two files

Answer. The screen shows **nothing**. `result.txt` contains the success line (`/etc/hostname`); `errors.txt` contains the message `ls: cannot access '/unknown-date': No such file or directory`. With `> result.txt 2>&1`, both lines would go into `result.txt` and `errors.txt` would not be created.

Why. `ls` writes its results to **stdout** (stream 1) and its complaints to **stderr** (stream 2). `>` only diverts stream 1, `2>` only stream 2; `2>&1` means "make stream 2 point where stream 1 points *right now*".

Nuance. Order matters: `2>&1 > result.txt` would send the errors... to the screen (stream 2 is plugged into the old stream 1, before the redirection). This separation of streams is what will let `podman logs` show you both a container's errors and its normal output (lab 03).

Example.

```
ls /etc/hostname /unknown-date > result.txt 2> errors.txt
cat result.txt      # /etc/hostname
cat errors.txt      # ls: cannot access '/unknown-date': No such file or directory
```

Question 9 — `127.0.0.1` vs `0.0.0.0`

Answer. Redis listens on `127.0.0.1:6379`: only the *loopback* interface, so reachable **only from the machine itself**. Java listens on `0.0.0.0:8080`: all interfaces, so reachable from the network. From another machine, only the Java service answers.

Why. The listening address is a filter: the kernel only hands the process the connections that arrived on that address. `0.0.0.0` means "all of the machine's addresses".

Nuance. This is a first-order security boundary: a database listening locally cannot be attacked from the network. In lab 07 you will see that `podman run -p 8080:80` publishes on `0.0.0.0` by default — and that `-p 127.0.0.1:8080:80` deliberately restricts. Understanding these two `ss` lines is already understanding `-p`.

Example.

```
python3 -m http.server 8080 --bind 127.0.0.1 &
ss -tlnp | grep 8080     # LISTEN ... 127.0.0.1:8080 ... ("python3",pid=...)
kill %1
```

Question 10 — Port 80 refused

Answer. Ports **below 1024** (so-called *privileged*) can only be opened by root (UID 0). Your process, UID 1000, is refused the `bind` on port 80; 8080 is above the threshold, hence free. Historically, the rule guaranteed that on a shared machine, an "official" service (port 25, 80...) could not be impersonated by a mere user. Consequence: rootless Podman, an ordinary process running under your UID, cannot publish `-p 80:80` — you publish `-p 8080:80` instead.

Why. The check is done by the kernel at the moment of the `bind` system call, based on the effective UID (more precisely, on a *capability* that root possesses).

Nuance. The threshold is tunable (`sysctl net.ipv4.ip_unprivileged_port_start`), and real production web servers run behind a load balancer which, itself, owns port 80. Podman's error message (`pasta failed ... Permission denied`) will come back in lab 07.

Example.

```
python3 -m http.server 80
# PermissionError: [Errno 13] Permission denied
python3 -m http.server 8080 &   # works
kill %1
```

Question 11 — `/proc`, the fake directory

Answer. `/proc` is a **virtual filesystem** (type `proc`), mounted on `/proc`, whose content exists on no disk: each read is fabricated on the fly by the kernel from its internal state. The numeric directories are the living processes; the other files describe the system. Usage examples: `/proc/<pid>/environ` (a process's real environment), `/proc/meminfo` (memory), `/proc/self/uid_map` (the UID mappings — the proof of rootless in lab 01).

Why. "Everything is a file": exposing the kernel's state as files lets you use `cat`, `grep` and `ls` as administration tools, without a dedicated API. `ps` is just a wrapper around `/proc`.

Nuance. This is also why a container receives **its own** `/proc` mounted at creation: otherwise it would see all the host's processes. When `ps` "lies" inside a container, it is because its `/proc` is isolated — not because the processes have disappeared.

Example.

```
findmnt -t proc          # /proc  proc  proc  rw,relatime
df -h /proc 2>/dev/null  # no disk associated
tr '\0' '\n' < /proc/self/environ | head -3   # the environment... of this very cat
```

Question 12 — `command not found`, code 127

Answer. The shell walked, in order, through every directory of the `PATH` variable looking for an executable `mytool`; `~/tools` is not in it, the search fails, code 127. Immediate: launch by explicit path, `~/tools/mytool`. Durable: (1) add the directory to the PATH in `~/.bashrc` (`export PATH="$HOME/tools:$PATH"`), or (2) copy/link the tool into a directory already present, such as `~/.local/bin` or `/usr/local/bin`.

Why. The PATH is the only resolution mechanism for "bare" commands. The current directory is deliberately not part of it: a booby-trapped `ls` dropped into `/tmp` must not run just because you happened to `cd /tmp`.

Nuance. 127 (not found) and 126 (found but not executable) are two distinct diagnoses. Inside a container, the error `exec: "mytool": executable file not found in $PATH` has exactly the same cause — the image's PATH (lab 04).

Example.

```
mkdir -p ~/tools && printf '#!/bin/bash\necho ok\n' > ~/tools/mytool && chmod +x ~/tools/mytool
mytool                   # command not found ; echo $? → 127
export PATH="$HOME/tools:$PATH"
mytool                   # ok
```

Question 13 — Why 12-factor loves the environment

Answer. Because the environment is attached to the **process**, not to the machine: it is fixed at launch, inherited automatically, and disappears with the process. For disposable, restartable applications, this yields a configuration that is (1) injectable from the outside without modifying either the code or the delivered files, (2) different per instance — two processes side by side with two configurations, (3) free of residual state: relaunching with other values is enough, nothing to clean up.

Why. Parent → child inheritance does all the work: whoever launches (the shell, systemd, later the container engine) prepares the dictionary, the application merely reads. The same artifact — JAR or image — moves from one environment to the next unchanged; only the set of variables changes.

Nuance. The immutability of the inheritance is also its limit: changing a variable requires **restarting** the process. That is a non-problem for a container (disposable by design), but real grief for whoever hoped to reconfigure live. Secrets, for their part, will deserve better than variables visible in `/proc/<pid>/environ` (lab 08).

Example.

```
SERVER_PORT=9090 java -jar app.jar   # same JAR, another port — nothing was modified
# which will become, word for word:
# podman run -e SERVER_PORT=9090 my-api
```